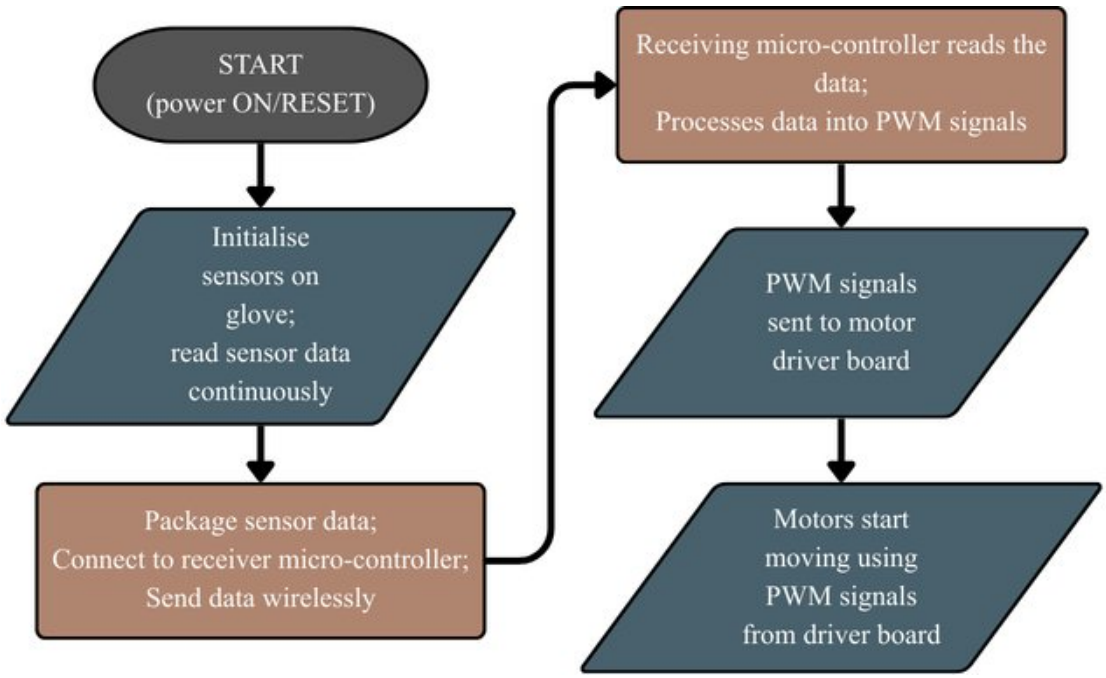
Contents:

1. General Working

We need to have the mode of communication between the transmitting end and the receiving end to be wireless so as to not provide hindrance to the user's arm movements.

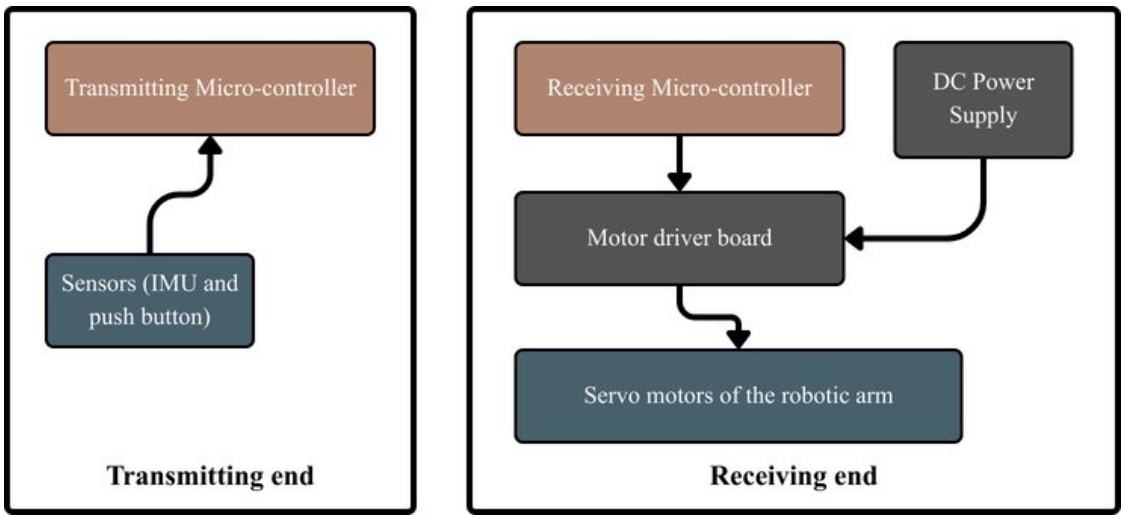
Our system realises this by capturing the user's arm movements via a glove that contains the required circuitry to capture the motion data, and have it sent to the micro-controller that can package this motion data efficiently and transmit it wirelessly to another micro-controller on the receiving end. On the receiving end, the micro-controller utilises this motion data and translates it into PWM signals that is fed to the motor driver board to control the robotic arm as desired. Hence, the system—consisting of the receiving end and the transmitting end—is realised as per the proposed design.

1.1. Flowchart diagram representation of general working



The system runs continuously as long as the micro-controllers are powered ON—system is reset by resetting the micro-controllers themselves.

1.2. Block Diagram illustratiom



This diagram provides a simple, visual overview of the functioning of the system in a simplified manner.

2. Components

- Micro-controller board used: ESP32 DOIT DEVKIT V1

This board was chosen over Arduino boards due to superior processing power, sufficient GPIO pins and especially due to built-in wireless communication features, making it a cheap and robust choice for this project.

- IMU Sensors used: MPU 6050

A popular budget IMU sensor which has integrated accelerometer and gyroscope. It was chosen for its ease of use and is sufficiently accurate for the requirements of this project.

- PCF8575:

The PCF8575 was considered to expand GPIO availability over I²C and support system scalability. However, due to identical sensor address limitations, a multiplexer provided a more appropriate solution.

- TCA9548A:

The TCA9548A multiplexer board was selected to manage multiple MPU6050 sensors with identical I²C addresses by isolating them across separate channels, enabling conflict-free communication over a single I²C bus in the ESP32.

- Flex Sensors:

The flex sensor was chosen to enable a continuously variable input signal that operates based on bending of the user's finger, which is a simple and intuitive input method for controlling the gripper motor of the robotic arm.

- Jumper wires:

Used to make the connections between the various components/boards, including connections made on the breadboard.

- DFRobot 60W Adjustable Buck Converter:

An old laptop charger was reused for providing power to the motors of the robotic arm, which can output 12V at a maximum amperage of 5A. Since the motor driver and the motors themselves operate at a voltage of 5V, and higher current, a buck converter is needed to regulate the laptop charger's power and provide stable power for the motors of the robotic arm.

This buck converter was chosen for its robust construction and adjustable output power.

- 18AWG Solid core insulated wire:

This wire was used to connect the buck converter to the motor driver board due to its low resistance which can safely handle high currents. Its thickness also provides good mechanical strength and is reliable for its purpose.

- Motor driver board used: PCA9685:

This is a very popular servo motor driver board that is often used to power servo motors that operate at 5V, which is commonly paired with Arduino and similar micro controllers via I^2C.

This board was selected for its ease of use and availability of abundant number of channels in an organised manner.

- 1000uF 10V Electrolytic Capacitors:

This capacitor was added across the supply line of the motor driver board to reduce voltage fluctuations caused by sudden current draw from the servo motors. Thus it improves the overall performance of the motors of the robotic arm.

- DIY Robotic Arm Kit (chassis):

A ready-made robotic arm chassis was selected for cost-effectiveness and reliability, allowing us to concentrate more on the electronics and coding for building this prototype.

- DIY Robotic Arm Kit (servo motors):

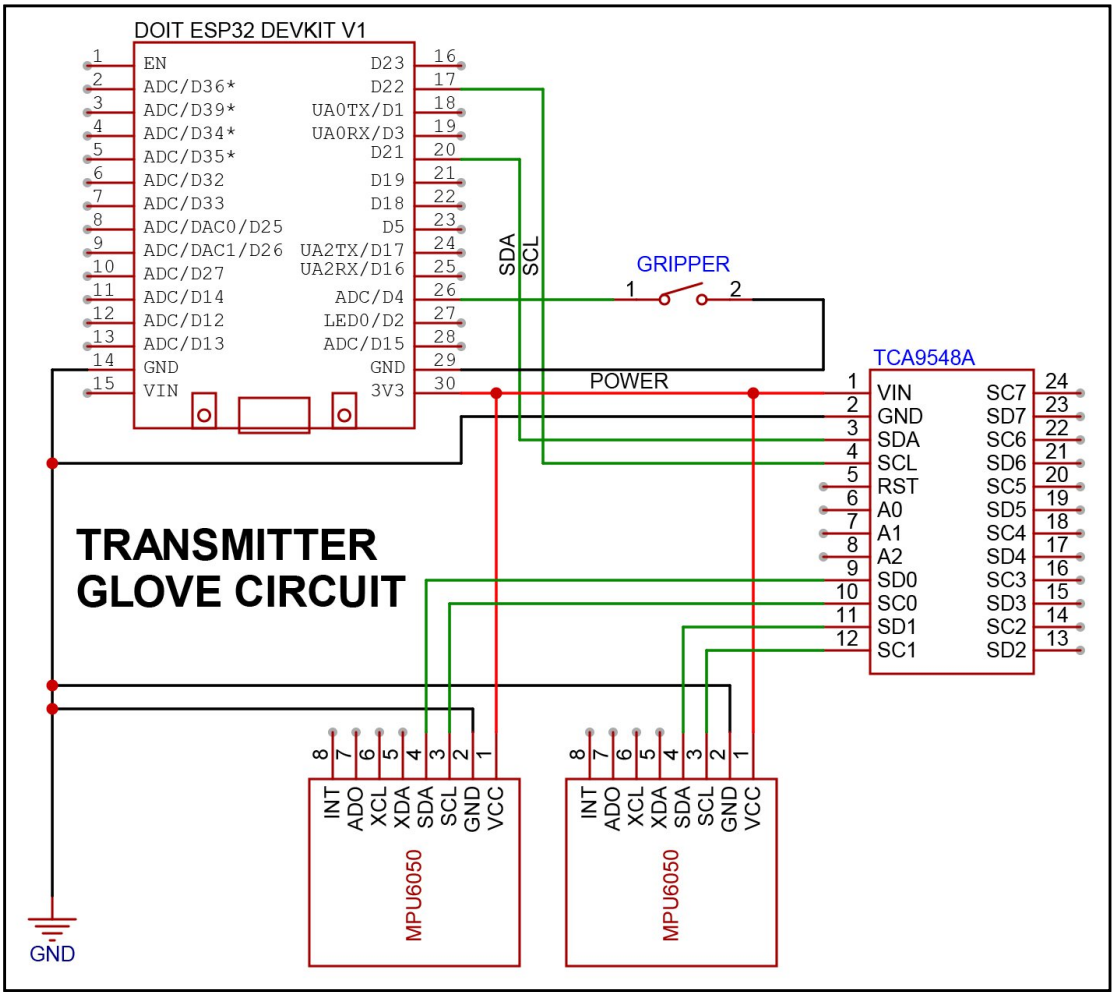
The servo motors included with the robotic arm kit were used as they are readily compatible with the chassis and provide sufficient torque and control required for the purpose of this project. The motors included with the kit were the MG996R and MG90S, both of which have s plastic gearbox.

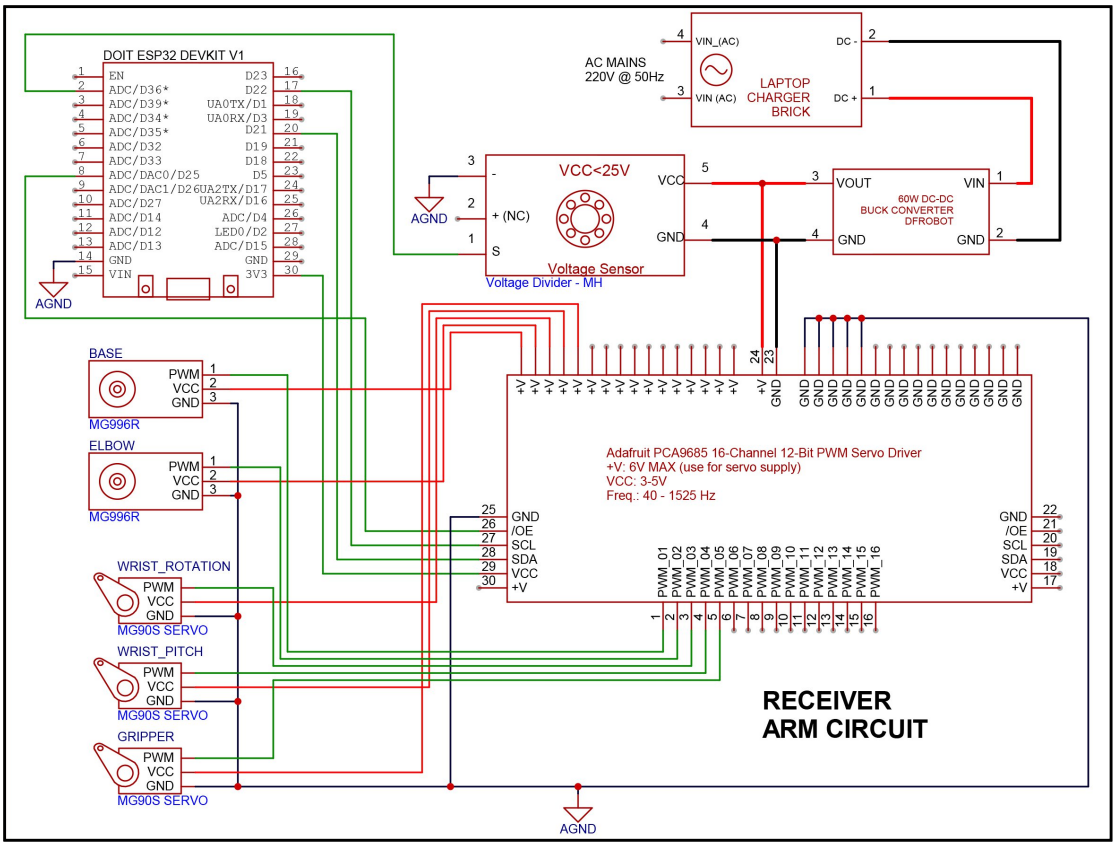
- Glove:

An arm sleeve glove is employed as a placeholder for the IMU sensors, TCA9548A multiplexer and the transmitting ESP32 micro-controller and other supporting components like jumper wires and breakout pins for power and grounding.

3. Circuitry

The circuit diagram of this project's set-up is as shown. Note that a switch is used as a substitute for the flex sensors to control the end effector—the reason for which will be disclosed later on in the Evaluation section.





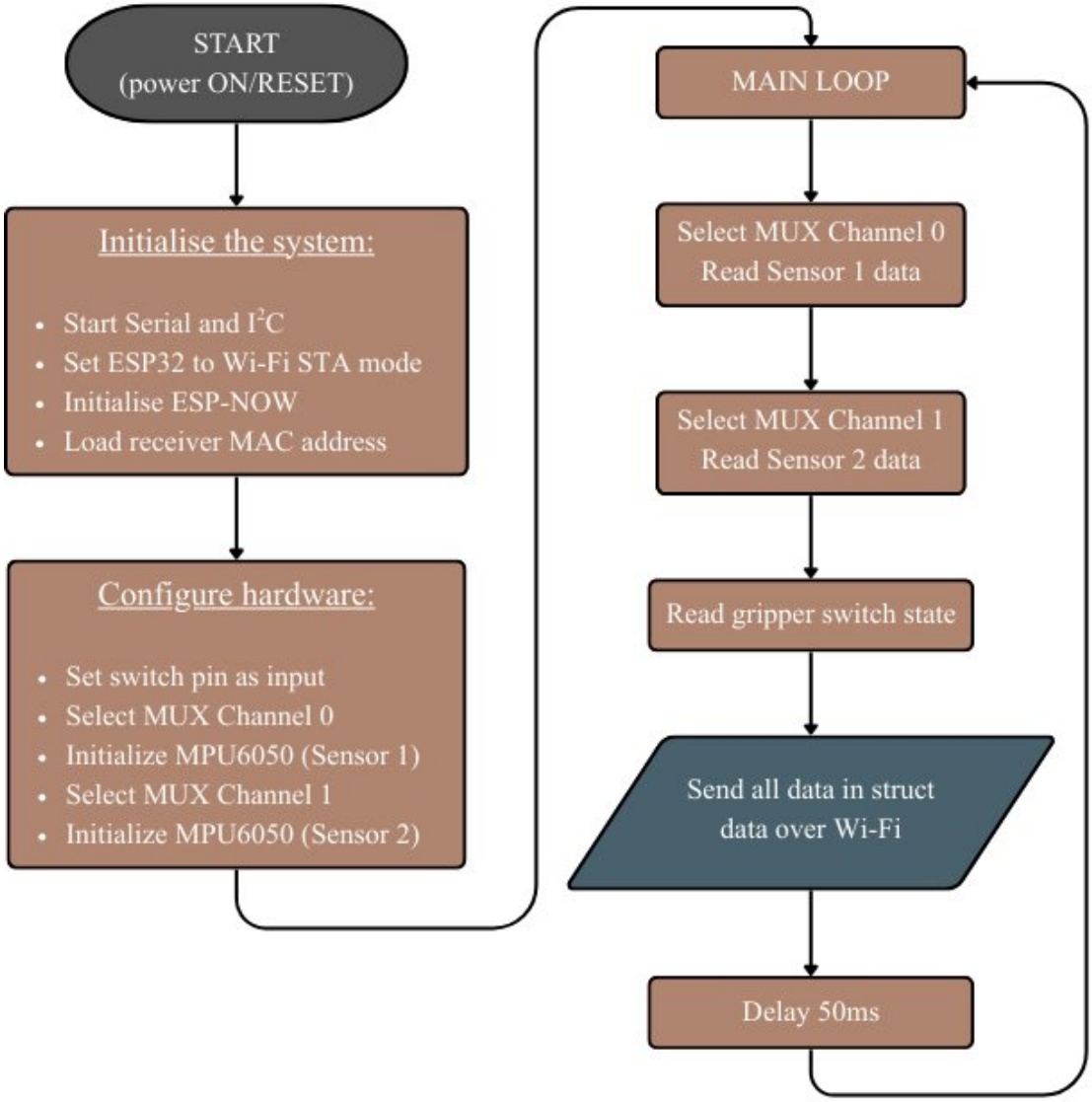
4. Code overview

The software for this system is divided between the two micro-controllers as—the transmitter and the receiver. C language has been used throughout.

4.1. Transmitter arm code

The ESP32 on the transmitting end is the one that reads the motion data from the MPU6050 sensors as well as the push button for gripper control. It does this by packaging the data into a struct datatype that is then sent over Wi-Fi using the ESP-NOW protocol. The main loop of the program here switches between the 2 channels of the multiplexer and sends the structured data with a delay between switching channels.

4.1.1. Flowchart of the transmitter glove code



This flowchart gives us the sequential operation of the transmitter code, which helps us to easily understand what the micro-controller is doing in real-time.

4.1.2. Explanations of the transmitter glove code

- struct datatypes:

struct datatype has been used to organise the sensors' data for streamlining all the required data, making it easier to handle throughout the program. We have defined 2 datatype structures:

1. SensorData—for holding the MPU6050 sensors' acceleration and gyroscopic data

2. MultiSensorData—for holding the SensorData of both IMUs as well as the state of the switch that controls the gripper's state. A single instance of this structure is used called allData that holds all the data that is to be sent to the robotic arm for operation.

- selectMuxChannel():

This function is used to multiplex the I2C communication, allowing the ESP32 to read the data from multiple MPU6050 sensors that share the same I2C address. Direct parallel communication is not possible due to the limitations of the I2C, so this function is used to switch between the sensors in the main loop() function during the operation.

- readMPU():

This function is called right after selecting a mux channel using its respective function. This function performs two roles:

1. Reads the sensor data in accordance to the struct datatype specified.

2. Also checks the acceleration and gyro data for its validity; i.e. if either one or both is NaN or not.

- setup():

The setup function is essentially the initialisation and configuration of all the hardware present. It also sets up the Wi-Fi connection to the ESP32 on the receiving end via the ESP-NOW protocol.

1. ESP-NOW protocol is used over conventional Wi-Fi and Bluetooth communication as it offers direct peer-to-peer communication, making it not only faster, but also more minimal and simpler to setup and run.

Two flags have been set-up—one that checks if the ESP32 on the transmitting side can initialise the protocol, and another that tries to connect to the receiver using its respective MAC address and returns if the connection has failed.

2. The switch for the gripper control has been initialised in the pinMode function.

3. Both the MPU6050 sensors are initialised and checked for availability—the ESP restarts if the sensors are not found. Additionally, some filtering is applied to the sensors themselves.

   These instructions configure the operating ranges and filtering characteristics of the MPU6050 sensor. The accelerometer range is set to balance sensitivity and motion tolerance, while the gyroscope range is configured to capture moderate to fast rotational movements in a balanced manner. A digital low-pass filter is applied to reduce high-frequency noise, ti improve signal stability during operation.

- loop():

This function is where the actual real-time operation of the transmitter system occurs. It selects each sensor, reads and checks the availability of the sensor data and updates the respective variables that hold the data, packaging it into the allData variable which is then sent over to the receiving end. A debug line is also available where the MPU data, if successfully sent, will serially print the raw data along with the state of the gripper. A small delay is added before the loop continues its next iteration.

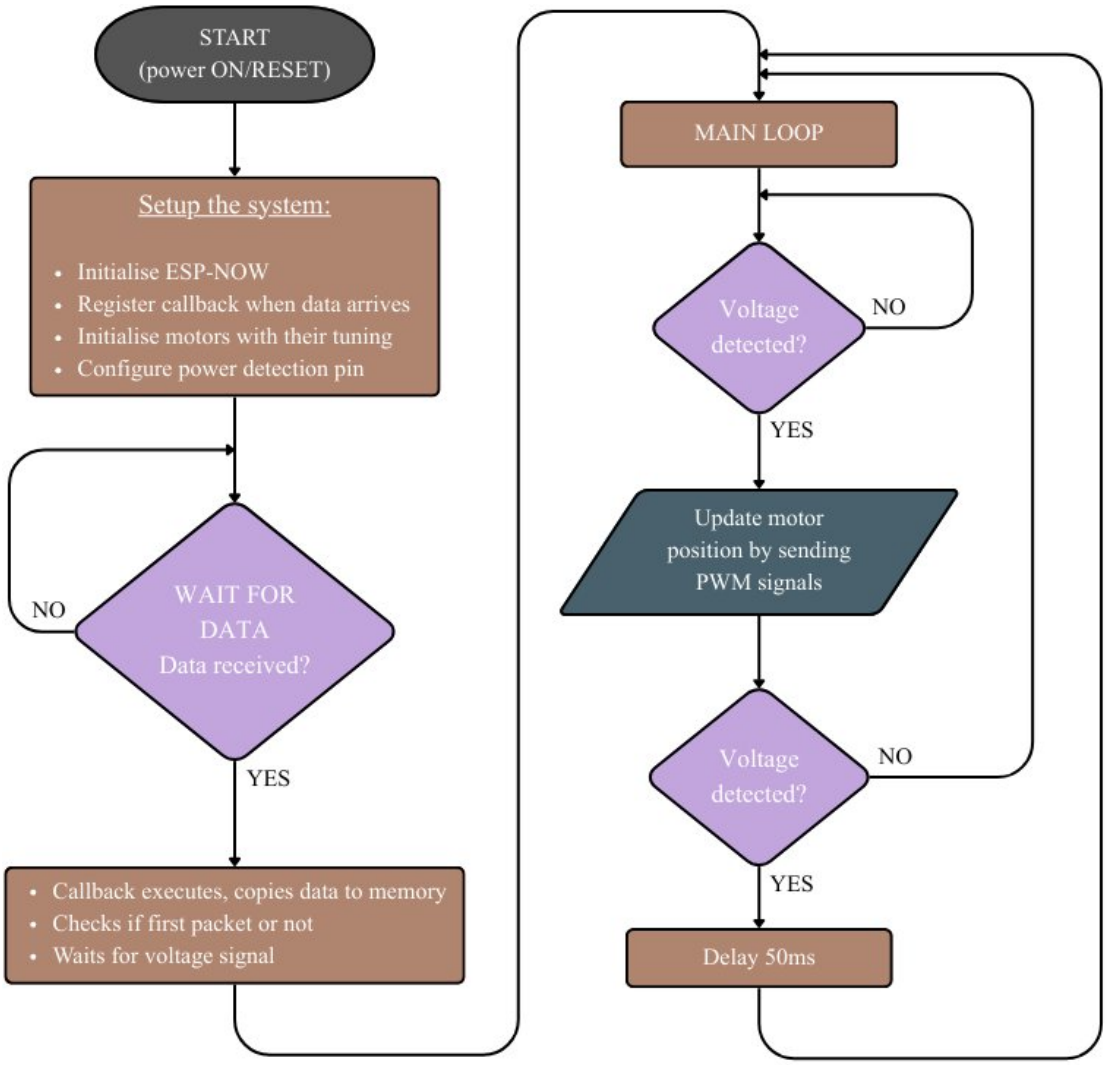
This concludes the operation of the transmitting glove's code which is responsible for aquiring and sending the data required for movement of the robotic arm.

4.2. Receiving arm code

The ESP32 on the receiving end is the one that controls the action of the motors that are responsible for the movements of the robotic arm. It receives the structured data from the transmitting ESP32 and validates that each packet has been received. Different functions are used for controlling each motor, which is called upon in the main loop function where the actual operation occurrs. An analog pin is used as an input to the ESP32 to test if the power for the servo motors is ON or OFF, based on which the operation stops/resumes. Once power is ON, the ESP32 starts translating the received data into PWM signals that are fed to the respective motors using the PCA9685 motor driver board.

The robotic arm is expected to be in its beginning state during initial operation, as servo motors have no feedback, which has its drawbacks which will be elaborated in the *Evaluation* section.

4.2.1. Flowchart of the receiving arm code



4.2.2. Explanations of receiving arm code

- Glocal variables:

1. Few variables are declared in the start of the program, which basically hold the configuration of the hardware we're using for this project.

2. The same data structure as that of the transmitter code is utilised here that will be the placeholder of the incoming data.

3. Certain other variables included under the “Motion tuning” section have been used to tune the overall behaviour of the motors themselves.

4. Another structured datatype has been used to package the various parameters of the motors that will be useful for their initialisation and updation during operation.

5. A boolean flag called “swapped” is used for tuning the axis swapping, which will be discussed later.

- OnDataRecv():

This function is a callback function the the ESP-NOW data reception. It is automatically executed whenever a data packet is received from the transmitting ESP32. First, the incoming data is copied to memory for processing. A flag is triggered that tells us that it is the first packet of data, and ensures all motors are aligned in their default position during initialisation. This ensures that the motors don’t suddenly jump into a position when the arm is powered ON, and only moves when there is change in the motion data. After this initial setup, this function continues to only update the data structure in the memory, which is processed later in other functions.

- Update Motor functions:

Each motor—base, elbow, wrist roll, wrist pitch and gripper—has its own update function that controls their respective movements that gives a total of 5 DOF.

Each function processes the latest received sensor data, applies motion tuning parameters such as deadzones and scaling, and converts the motion values into appropriate PWM signals for the motor driver. Gradual position updates are used instead of sudden jumps, ensuring smooth and stable motion.

Special handling is implemented for the base and elbow motors through an axis swapping mechanism. Depending on the orientation of the IMU sensors, the

relevant gyroscopic axes are dynamically swapped, which is identified and triggered using a boolean control flag. This corrects directional inconsistencies caused by sensor mounting orientation and ensures intuitive arm movement.

This approach ensures that the control logic of each motor is isolated within their respective functions and the main loop is executed in an orderly manner.

- setup():

This function initialises all the hardware components and the communication protocol required to run the system.

1. The ESP-NOW protocol is initialised to start wireless peer-to-peer communication with the transmitting ESP32. The receiver is registered to trigger the OnDataRecv() function each time a data packet is received when connection is successful.

2. Each motor is initialised and is held at their default positions, and waits until the servo power ON condition is satisfied, ensuring that no signals are unnecessarily sent to the motors, holding the enable pin of the PCA9685 HIGH until then. This finishes the initial setup of the robotic arm.

- main loop():

This function runs the robotic arm continuously and handles the operation of it in real-time. It repeatedly processes the latest received motion data and gripper state, updating the motors accordingly.

It makes sure to wait until the first packet of data is received, and also reads the status of the analog input pin, ensuring that servo motor power is ON; so that PWM signals can be sent to the PCA9685 by pulling the enable pin LOW.

The gripper motor is not run continuously so that power is not wasted in running it continuously when open and closed.

At the end of each iteration, it makes sure to check once more for the detection of the servo motor power, skipping all the code if the power is OFF. The loop continues with a small delay.

This concludes the operation of the receiving robotic arm's code, which is responsible for actuating the robotic arm's motors and translating the motion data of the user's arm into a tangible mechanical movement.

5. Evaluation

The robotic arm system successfully achieved real-time motion tracking and wireless control using MPU6050 sensors and ESP-NOW wireless communication. The arm was able to replicate the user's arm movements with minimal latency, demonstrating effective sensor integration and motor coordination.

Smooth motor operation was achieved through gradual position updates, deadzone tuning, and motion scaling, which significantly reduced jitter and unintended movements. The axis swapping logic further improved intuitive control by compensating for sensor orientation differences.

However, several limitations were observed during operation. Sensor drift over time caused minor positional inaccuracies, requiring periodic recalibration i.e. resets. Additionally, the lack of positional feedback from the servo motors meant that the system assumed a fixed initial reference position, which could lead to cumulative errors after prolonged use.

During implementation, one joint of the robotic arm was removed to improve mechanical stability and simplify control, as the additional degree of freedom introduced unnecessary complexity and reduced reliability. This adjustment allowed smoother and more controllable operation of the remaining joints.

Additionally, the originally planned flex sensor for gripper control was replaced with a push-button switch. This change was made to improve responsiveness and eliminate inconsistent readings caused by sensor bending variations from wear and tear, as the ones bought were not reliable enough, resulting in a much more reliable gripper actuation.

Mechanical limitations of the chassis and motor load also affected the stability of the base joint under heavier stress. But, for a prototype of this caliber, it clearly demonstrates the potential of the system as envisioned.

Overall, while this prototype system performed reliably for real-time gesture-based control, improvements such as closed-loop motor feedback and enhanced sensor filtering could further increase precision and long-term stability.